

Парадигмы программирования

Парадигмы программирования

Парадигма программирования (подход к программированию) — совокупность идей и понятий, определяющих стиль написания компьютерных программ.

Парадигма программирования определяется:

- вычислительной моделью;
- базовой программной единицей (-ами);
- методами разделения абстракций.

Основные парадигмы программирования:

- императивное;
- структурное;
- объектно-ориентированное;
- функциональное;
- логическое.

Императивное программирование

Вычислительная техника создавалась для решения математических задач — расчета баллистических траекторий, численного решения уравнений и т.д.



Первые языки программирования, такие как Fortran, Алгол, реализованны в парадигме императивного программирования (ИП).

Императивное программирование

Основные механизмы управления в ИП:

- последовательное исполнение команд;
- использование именованных переменных;
- использование оператора присваивания;
- использование ветвления (оператор if);
- использование безусловного перехода (оператор goto).

Ключевой идеей императивного программирования является работа с переменными, как с временным хранением данных в оперативной памяти.



В основе императивных языков программирования лежит [машина Тьюринга](#), разработанная [Аланом Тьюрингом](#).

Логическое программирование

При использовании логического программирования программа содержит описание проблемы в терминах фактов и логических формул, а решение проблемы система находит с помощью механизмов логического вывода.

В конце 60-х годов XX века Корделл Грин предложил использовать резолюцию как основу логического программирования. Алан Колмеро создал язык логического программирования [Prolog](#) в 1971 году.

Логическое программирование



- достаточно высокий уровень машинной независимости, а также возможность откатов, возвращения к предыдущей подцели при отрицательном результате анализа одного из вариантов в процессе поиска решения;



- специфичность класса решаемых задач;
- сложность эффективной реализации для принятия решений в реальном времени;

Функциональное программирование

Основной инструмент функционального программирования (ФП) — математические функции.

Функциональная программа — набор определений функций. Функции определяются через другие функции или рекурсивно через самих себя. При выполнении программы функции получают аргументы, вычисляют и возвращают результат, при необходимости вычисляя значения других функций.



В основе функциональных языков программирования лежит модель лямбда-исчислений, разработанная [Алонзо Чёрчем](#).

Основные идеи функционального программирования:

- **неизменяемые переменные** — можно определить переменную, но изменить ее значение нельзя;
- **чистая функция** — это функция, результат работы которой предсказуем. При вызове с одними и теми же аргументами, такая функция всегда вернет одно и то же значение;
- **функции высшего порядка** — могут принимать другие функции в качестве аргумента или возвращать их;
- **рекурсия** — поддерживается многими языками программирования, а для функционального программирования обязательна;
- **лямбда-выражения** — способ определения анонимных функциональных объектов.

Структурное программирование

Структурная парадигма программирования нацелена на сокращение времени разработки и упрощение поддержки программ за счёт использования блочных операторов и подпрограмм. Отличительная черта структурных программ — отказ от оператора безусловного перехода (goto).

Основные механизмы управления:

- последовательное исполнение команд;
- использование именованных переменных;
- использование оператора присваивания;
- использование ветвления (оператор if);
- использование циклов;
- использование подпрограмм (функций).



Парадигму структурного программирования предложил нидерландский ученый Эдсгер Дейкстра.

Объектно-ориентированное программирование

В объектно-ориентированной парадигме программа разбивается на объекты – структуры данных, состоящие из полей, описывающих состояние, и методов – функций, применяемых к объектам для изменения или запроса их состояния.

Основные механизмы управления:

- абстракция;
- класс;
- объект;
- полиморфизм;
- инкапсуляция;
- наследование.



В основе императивных, структурных, объектно-ориентированных языков программирования лежит [машина Тьюринга](#), разработанная [Аланом Тьюрингом](#).

Парадигма программирования в Python

Язык программирования не обязательно использует единственную парадигму. Существуют мультипарадигменные языки. Создатели таких языков считают, что ни одна парадигма не может быть одинаково эффективной для всех задач, и следует позволять программисту выбирать лучшую для решения каждой.



Язык Python мультипарадигменный.

Зачем нужно что-то новое?

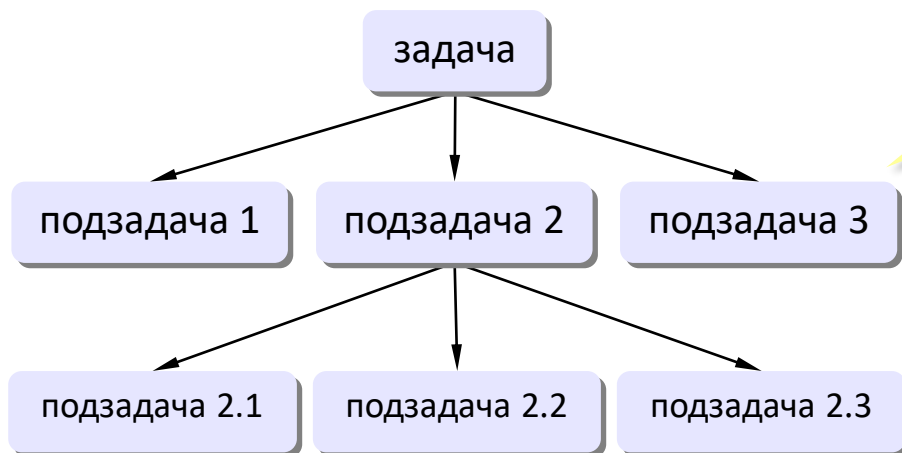


Главная проблема – **сложность!**

- программы из миллионов строк
- тысячи переменных и массивов

Э. Дейкстра: «Человечество еще в древности придумало способ управления сложными системами: **«разделяй и властвуй»**».

Структурное программирование:



декомпозиция по задачам



человек мыслит иначе, объектами

Объект класса



состояние

методы

Cat

энергия

настроение

есть

спать

?

Можно изменять
вручную?

метод **есть**

+ энергия

+ настроение

- голод

метод **спать**

+ энергия

+ голод

метод **играть**

- энергия

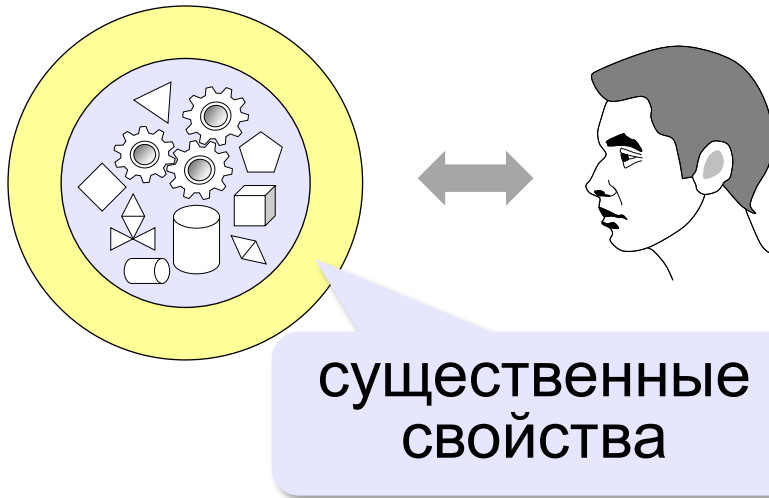
+ настроение

+ голод

!

Меняем состояние
только через методы!

Как мы воспринимаем объекты?



Абстракция – это выделение существенных свойств объекта, отличающих его от других объектов.



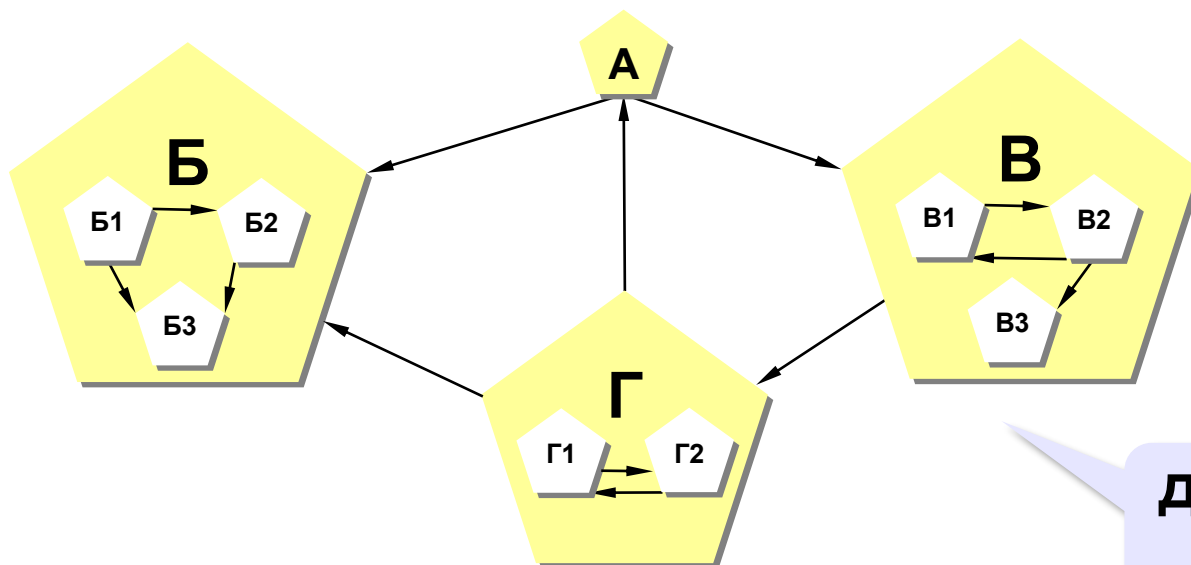
Разные цели –
разные модели!

Использование объектов

Программа – множество объектов (моделей), каждый из которых обладает своими свойствами и поведением, но его внутреннее устройство скрыто от других объектов.



Нужно «разделить» задачу на объекты!



**декомпозиция по
объектам**

Функциональное программирование в Python

Функциональное программирование в Python

Функциональное программирование представляет собой методику написания программного обеспечения, в центре внимания которой находятся функции.

Функции могут присваиваться переменным, они могут передаваться в другие функции и порождать новые функции.

Python имеет богатый и мощный арсенал инструментов, которые облегчают разработку функционально-ориентированных программ.

Позиционные аргументы функции

Аргументы функции, которые передаются без указания имен, называются **ПОЗИЦИОННЫМИ**.

По позиции, расположению аргумента, функция «понимает», какому параметру он соответствует.

Рассмотрим следующий код:

```
def diff(x, y):  
    return x - y  
# используем позиционные аргументы  
res = diff(10, 3)  
print(res)
```



При вызове функции `diff()` **первому** параметру `x` будет соответствовать **первый** переданный аргумент, 10, а **второму** параметру `y` — **второй** аргумент, 3.

Именованные аргументы функции

Аргументы, передаваемые с именами, называются **именованными**. При вызове функции можно использовать имена параметров из ее определения.

Рассмотрим следующий код:

```
def diff(x, y):  
    return x - y  
  
# используем именованные аргументы  
res = diff(x=10, y=3)  
print(res)
```



При вызове функции `diff()` мы явно указываем, что параметру `x` соответствует аргумент `10`, а параметру `y` — аргумент `3`.

Именованные аргументы функции

Использование именованных аргументов позволяет нарушать их позиционный порядок при вызове функции. Порядок упоминания именованных аргументов не имеет значения!

Мы можем вызвать функцию `diff()` так:

```
res = diff(y=3, x=10)
```



В соответствии с PEP 8 при указании значений именованных аргументов при вызове функции знак равенства не окружается пробелами.

Именованные аргументы функции



Когда стоит применять именованные аргументы?

Если функция принимает больше трёх аргументов, нужно хотя бы часть из них указать по имени. Особенно важно именовать значения аргументов, когда они относятся к одному типу, ведь без имен очень трудно понять, что делает функция с подобным вызовом.

Именованные аргументы функции

Рассмотрим определение функции `make_circle()` для рисования круга:

```
def make_circle(x, y, radius, line_width, fill):  
    # тело функции
```

Вызвать такую функцию можно так:

```
make_circle(200, 300, 17, 2.5, True)
```

Или так:

```
make_circle(x=200, y=300, radius=17,  
            line_width=2.5, fill=True)
```



Второй вызов читать значительно проще!

Именованные аргументы функции

Когда значение аргументов очевидно, можно их не именовать.



Что скрывается за значениями при вызове следующих функций?

```
point3d(7, 50, 13)
```

```
rgb(7, 255, 45)
```

координаты
точки в
трехмерном
пространстве

значения компонент **red**, **green**, **blue** некоторого цвета

Именованные аргументы функции

? Какие именованные аргументы можно использовать для функции **print**?

Именованными аргументами функции **print** являются **sep** и **end**

Например:

```
print('eeee', 'ffff', sep='123', end='python')
```

Результат:
eeee123ffffpython

? Что получим?

Комбинирование позиционных и именованных аргументов

Рассмотрим вызовы функции:



Что получим?

```
def diff(x, y):  
    return x - y  
res = diff(10, y=3)  
print(res)
```

Результат:
7

```
def diff(x, y):  
    return x - y  
res = diff(x=10, 3)  
print(res)
```

Результат:
SyntaxError:
positional argument
follows keyword
argument



Позиционные значения должны быть указаны **до** любых именованных!

Необязательные аргументы функции

Бывает, что какой-то параметр функции часто принимает одно и то же значение.

Например, функция `int()`, преобразующая строку в число, принимает два аргумента:

- первый аргумент: строка, которую нужно преобразовать в число;
- второй аргумент: основание системы счисления.

```
num = int('101', 2)  
# аргумент 2 указывает на то, что число 101  
# записано в двоичной системе
```

Необязательные аргументы функции

Но чаще всего функция `int()` используется для считывания из строки чисел, записанных в десятичной системе счисления.

В таких ситуациях Python позволяет задавать некоторым параметрам **значения по умолчанию**. У функции `int()` второй параметр по умолчанию равен 10, и потому можно вызывать эту функцию с одним аргументом. Значение второго подставится автоматически.

```
num = int('101')
```

Необязательные аргументы функции



Как задать значение параметра по умолчанию?

Чтобы задать значение параметра по умолчанию, в списке параметров функции достаточно после имени переменной написать знак равенства и нужное значение.

Параметры со значением по умолчанию идут последними, ведь иначе интерпретатор «не смог бы понять», какой из аргументов указан, а какой пропущен.

Рассмотрим определение функции `make_circle()`:

Поскольку обычно нам нужно рисовать круг с шириной линии, равной 1 с заливкой, то логично установить данные значения в качестве значений по умолчанию:

```
def make_circle(x, y, radius,  
                line_width=1, fill=True):  
    # тело функции
```

Функции с переменным количеством аргументов

Рассмотрим определение функции `my_func()`:

```
def my_func(*args):  
    print(type(args))  
    print(args)  
my_func()  
my_func(1, 2, 3)  
my_func('a', 'b')
```



Звездочка в определении функции означает, что параметр `args` получит в виде кортежа все аргументы, переданные в функцию при ее вызове от текущей позиции и до конца.

Приведенный выше код выводит:

```
<class 'tuple'>  
( )  
<class 'tuple'>  
(1, 2, 3)  
<class 'tuple'>  
( 'a', 'b' )
```

Функции с переменным количеством аргументов

При описании функции можно использовать **только один** параметр помеченный звездочкой, причем располагаться он должен **в конце списка параметров**.

Например:

```
def my_func(num, *args):  
    print(args)  
    print(num)
```

```
my_func(17, 'Python', 2, 'C#')
```

Результат:



Что получим?

```
('Python', 2, 'C#')  
17
```

Функции с переменным количеством аргументов

Помеченный звездочкой параметр `*args` принимает отсутствие аргументов, в то время как для позиционных параметров аргументы всегда обязательны.

Например:

```
def my_func(num, *args):  
    print(args)  
    print(num)
```

```
my_func(17)
```

Результат:

```
()  
17
```



Что получим?



Функция `my_func()` принимает несколько аргументов, но как минимум один аргумент должен быть передан обязательно!

Передача аргументов в форме списка и кортежа

В некоторых случаях необходимо сначала сформировать набор аргументов, а потом передать их функции.

Например:

```
sum1 = sum([1, 2, 3, 4])
sum2 = sum((10, 20, 30, 40))
sum3 = sum(1, 2, 3, 4)
print(sum1)
print(sum2)
print(sum3)
```



Что получим?

Результат:

```
10
100
TypeError: sum expected at most 2 arguments, got 4
```

Передача аргументов в форме списка и кортежа

Используем оператор распаковки коллекций, который обозначается звездочкой *

Напишем свою версию функции `sum()`, функцию `my_sum()`, которая принимает переменное количество аргументов и вычисляет их сумму:

```
def my_sum(*args):  
    return sum(args) # args - это кортеж  
  
print(my_sum())  
print(my_sum(1))  
print(my_sum(1, 2))  
print(my_sum(1, 2, 3, 4))
```

Результат:

0
1
3
10



Что получим?

Передача аргументов в форме списка и кортежа

Можем вызвать функцию `my_sum()`, передавая ей списки или кортежи, предварительно распаковав их.

```
# распаковка списка  
print(my_sum(*[1, 2, 3, 4, 5]))
```

```
# распаковка кортежа  
print(my_sum(*(1, 2, 3)))
```

Результат:
15
6

Часть аргументов можно передавать непосредственно и даже коллекции подставлять не только по одной

```
print(my_sum(1, 2, *[3, 4, 5], *(7, 8, 9), 10))
```

Результат:
49

Получение именованных аргументов в виде словаря

Позиционные аргументы можно получать в виде `*args`, причём произвольное их количество. Такая возможность существует и для именованных аргументов. Только именованные аргументы получаются в виде словаря, что позволяет сохранить имена аргументов в ключах.

Получение именованных аргументов в виде словаря

Например:

```
def my_func(**kwargs):  
    print(type(kwargs))  
    print(kwargs)  
my_func()  
my_func(a=1, b=2)  
my_func(name='Timur', job='Teacher')
```

Результат:

```
<class 'dict'>  
{}  
<class 'dict'>  
{'a': 1, 'b': 2}  
<class 'dict'>  
{'name': 'Timur', 'job': 'Teacher'}
```

Получение именованных аргументов в виде словаря

Параметр ****kwargs** пишется в самом конце, после последнего аргумента со значением по умолчанию. При этом функция может содержать и ***args** и ****kwargs** параметры.

```
def my_func(a, b, *args, name='Gvido',  
            age=17, **kwargs):  
    print(a, b)  
    print(args)  
    print(name, age)  
    print(kwargs)  
my_func(1, 2, 3, 4, name='Timur', age=28,  
        job='Teacher', language='Python')
```



Что получим?

Результат:

1 2

(3, 4)

Timur 28

{'job': 'Teacher', 'language': 'Python'}

Получение именованных аргументов в виде словаря

```
def my_func(a, b, *args, name='Gvido',
            age=17, **kwargs):
    print(a, b)
    print(args)
    print(name, age)
    print(kwargs)
my_func(1, 2, name='Timur', age=28,
        job='Teacher', language='Python')
```



Что получим?

Результат:

1 2

()

Timur 28

{'job': 'Teacher', 'language': 'Python'}

Получение именованных аргументов в виде словаря

```
def my_func(a, b, *args, name='Gvido' ,  
            age=17, **kwargs):  
    print(a, b)  
    print(args)  
    print(name, age)  
    print(kwargs)  
my_func(1, 2, 3, 4, job='Teacher' ,  
        language='Python')
```



Что получим?

Результат:

1 2

(3, 4)

Gvido 17

{'job': 'Teacher', 'language': 'Python'}

Передача именованных аргументов в форме словаря

Как и в случае позиционных аргументов, именованные можно передавать в функцию "пачкой" в виде словаря. Для этого нужно перед словарём поставить две звёздочки.

Рассмотрим определение функции `my_func()`:

```
def my_func(**kwargs):  
    print(type(kwargs))  
    print(kwargs)  
info = {'name': 'Timur', 'age': '28',  
        'job': 'teacher'}  
  
my_func(**info)
```

Результат:
<class 'dict'>
{'name': 'Timur', 'age': '28', 'job': 'teacher'}

Передача именованных аргументов в форме словаря

Рассмотрим пример определения функции `print_info()`, распечатающей информацию о пользователе.

```
def print_info(name, surname, age, city,
               *children, **additional_info):
    print('Имя:', name)
    print('Фамилия:', surname)
    print('Возраст:', age)
    print('Город проживания:', city)
    if len(children) > 0:
        print('Дети:', ', '.join(children))
    if len(additional_info) > 0:
        print(additional_info)
```

Передача именованных аргументов в форме словаря

```
def print_info(name, surname, age, city,
               *children, **additional_info):
    print('Имя:', name)
    print('Фамилия:', surname)
    print('Возраст:', age)
    print('Город проживания:', city)
    if len(children) > 0:
        print('Дети:', ', '.join(children))
    if len(additional_info) > 0:
        print(additional_info)
```

```
children = ['Бодхи Грин', 'Ноа Грин', 'Джорни Грин']
additional_info = {'height': 163, 'job': 'actress'}
```

```
print_info('Меган',
           *children, **additional_info)
```

Результат:

Имя: Меган

Фамилия: Фокс

Возраст: 34

Город проживания: Ок-Ридж

Дети: Бодхи Грин, Ноа Грин, Джорни Грин

{'height': 163, 'job': 'actress'}

Keyword-only аргументы

В Python есть возможность пометить именованные аргументы функции так, чтобы вызвать функцию можно было, только передав эти аргументы по именам. Такие аргументы называются **keyword-only** и их нельзя передать в функцию в виде позиционных.

Рассмотрим определение функции `make_circle()`:

```
def make_circle(x, y, radius, *,  
                line_width=1, fill=True):
```



Здесь `*` выступает разделителем: отделяет обычные аргументы (их можно указывать по имени и позиционно) от строго именованных.

Keyword-only аргументы

```
def make_circle(x, y, radius, *,  
               line_width=1, fill=True):
```

```
make_circle(10, 20, 5) # (1)
```

✓

```
make_circle(10, 20, 15, 20) # (2)
```

✓

```
make_circle(x=10, y=20, 15, True) # (3)
```

```
make_circle(x=10, y=20, radius=7) # (4)
```

```
make_circle(10, 20, radius=10, line_width=2,  
            fill=False) # (5)
```

✓

```
make_circle(10, 20, 10, 2, False) # (6)
```

```
make_circle(x=10, y=20, radius=17,  
            line_width=3) # (7)
```



Какие из вызовов функций приведут к возникновению ошибок?

Keyword-only аргументы

Можно объявить функцию, у которой будут только строго именованные аргументы, для этого нужно поставить звёздочку в самом начале перечня аргументов.

```
def make_circle(*, x, y, radius,  
                line_width=1, fill=True):
```

Теперь для вызова функции `make_circle()` нам нужно передать значения всех аргументов явно через их имя:

```
make_circle(x=10, y=20, radius=15)  
make_circle(x=10, y=20, radius=15,  
            line_width=4, fill=False)
```



Разделитель `*` можно использовать только один раз в определении функции. Его нельзя применять в функциях с неограниченным количеством позиционных аргументов `*args`.

Примечания.

Примечание 1.

Специальный синтаксис ***args** и ****kwargs** в определении функции позволяет передавать функции переменное количество позиционных и именованных аргументов.

При этом args и kwargs **просто имена**. Вы не обязаны их использовать, можно выбрать любые, однако среди Python программистов приняты именно эти.

Примечания.

Примечание 2. Вы можете использовать одновременно `*args` и `**kwargs` в одной строке для вызова функции. В этом случае порядок имеет значение. Как и аргументы, не являющиеся аргументами по умолчанию, `*args` должны предшествовать и аргументам по умолчанию, и `**kwargs`. Правильный порядок параметров:

1. позиционные аргументы,
2. `*args` аргументы,
3. `**kwargs` аргументы.

```
def my_func(a, b, *args, **kwargs):
```


Функции как объекты

Функции как объекты

Функции - это совершенно отдельный элемент языка со своим синтаксисом и механизмом работы.

Но, оказывается, функции также что-то вроде особого типа объектов.

Бывают числа, строки, списки, кортежи, словари, множества. А бывают — функции. У каждого из этих типов есть свои операции, свой синтаксис, но все они — объекты.



Все в Python - объект, в том числе и функции!

Функции как объекты

Рассмотрим следующий код:

```
num = 17
numbers = [1, 2, 3]
colors = (1, 2, 3)
name = 'Python'

print(type(num))
print(type(numbers))
print(type(colors))
print(type(name))
print(type(print))
print(type(sum))
print(type(abs))
```



Python выдает нам строки которые поясняют, что **print**, **sum**, **abs** — встроенные функции языка, или методы.

Результат:

```
<class 'int'>
<class 'list'>
<class 'tuple'>
<class 'str'>
<class 'builtin_function_or_method'>
<class 'builtin_function_or_method'>
<class 'builtin_function_or_method'>
```

Функции как объекты

Рассмотрим следующий код:

```
num = 17
numbers = [1, 2, 3]
colors = (1, 2, 3)
name = 'Python'

print(type(num))
print(type(numbers))
print(type(colors))
print(type(name))
print(type(print))
print(type(sum))
print(type(abs))
```



Внимание: скобки не ставим при передаче аргумента в функцию `type()`, мы не вызываем функцию, а передаем ее название.

Результат:

```
<class 'int'>
<class 'list'>
<class 'tuple'>
<class 'str'>
<class 'builtin_function_or_method'>
<class 'builtin_function_or_method'>
<class 'builtin_function_or_method'>
```

Функции как объекты

Объявим свою функцию:

```
def hello():  
    print('Hello from function')  
  
print(type(hello))
```

Результат:
<class 'function'>

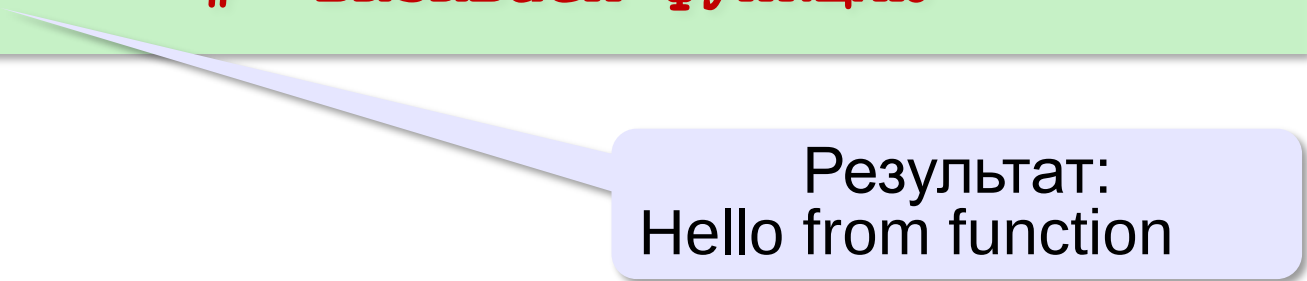


Поскольку функции тоже объекты, работать с ними можно и как с объектами: записывать их в переменные, передавать в качестве аргументов другим функциям, возвращать из функций и т.д.

Функции как объекты

Например:

```
def hello():  
    print('Hello from function')  
  
# присвоим переменной func функцию hello  
func = hello  
func()                # вызываем функцию
```

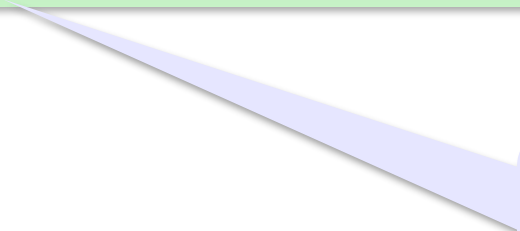


Результат:
Hello from function

Функции как объекты

Если по какой-либо причине вам не нравится название встроенной функции `print()`, можно написать такой код:

```
writeln = print           # как в языке Pascal  
  
writeln('Hello world! ')  
writeln('Python')        # вызываем функцию
```



Результат:
Hello world!
Python

Функции как объекты

Возможность записать функцию в переменную позволяет гибко управлять тем, какую функциональность мы хотим использовать. В одну и ту же переменную можно записать разные варианты поведения и менять их при необходимости. При этом не только не нужно будет менять код по всей программе, но и не придется даже изменять код функций. Достаточно переменной присвоить вместо одной функции — другую.

Функции как объекты

Например, необходимо выполнить некую функцию, если задано имя команды. Предположим, если пришла команда `start` — надо выполнить функцию `start()`, если команда `stop` — функцию `stop()`, если команда `pause` — функцию `pause()`.

Такую логику легко можно написать при помощи условного оператора `if .. elif`.

Функции как объекты

```
def start():  
    # тело функции start  
def stop():  
    # тело функции stop  
def pause():  
    # тело функции pause  
  
# считываем название команды  
command = input()  
if command == 'start':  
    start()  
elif command == 'stop':  
    stop()  
elif command == 'pause':  
    pause()
```



Если команд будет много или, если их количество будет увеличиваться, то оператор if получится слишком громоздким.

Функции как объекты

Приведенный ниже код решает ту же самую задачу:

```
def start():  
    # тело функции start  
def stop():  
    # тело функции stop  
def pause():  
    # тело функции pause
```



Такой код намного гибче!

```
# словарь соответствия команда → функция  
commands = {'start': start,  
            'stop': stop,  
            'pause': pause}
```

```
command = input() # считываем название команды  
commands[command]() # вызываем нужную функцию  
# через словарь по ключу
```

Встроенные функции, принимающие функции в качестве аргументов

Возможность присваивать имя функции переменной позволяет передавать имя функции аргументом другой функции.

Пример 1:

```
numbers = [8, -100, -50, 32, 87, 117, -210]
print(max(numbers))
print(min(numbers))
print(sorted(numbers))
```

Результат:

```
117
-210
[-210, -100, -50, 8, 32, 87, 117]
```



А если нужно найти **максимальный по модулю** элемент списка numbers?

Встроенные функции, принимающие функции в качестве аргументов

На этот случай встроенные функции `min()`, `max()`, `sorted()` могут принимать необязательный аргумент `key` – функцию, определяющую условия сравнения элементов. Другими словами, значение `key` должно быть функцией, принимающей один аргумент и возвращающей на его основе ключ для сравнения.



Встроенные функции `min()`, `max()`, `sorted()` – функции высшего порядка, так как принимают в качестве аргумента функцию сравнения элементов!

Функция, определяющая условия сравнения элементов, называется **компаратор** (compare – сравнивать).

Встроенные функции, принимающие функции в качестве аргументов

Продemonстрируем вышесказанное на примере кода:

```
numbers = [8, -100, -50, 32, 87, 117, -210]

# указываем функцию abs в качестве компаратора
print(max(numbers, key=abs))
print(min(numbers, key=abs))
print(sorted(numbers, key=abs))
```

Результат:

-210

8

[8, 32, -50, 87, -100, 117, -210]



Найден максимальный и минимальный элемент по модулю и произведена сортировка по модулю элементов списка numbers

Встроенные функции, принимающие функции в качестве аргументов

Пример 2:

Пусть в списке `points` хранятся в виде кортежей координаты точек плоскости в двумерной системе координат.

```
points = [(-10, 15), (2, -4), (2, 2), (10, 9)]
```

При использовании списочного метода `sort()` сортировка пройдет по первым значениям пар кортежа, а в случае их совпадения, по вторым.

Встроенные функции, принимающие функции в качестве аргументов

Таким образом, приведенный ниже код:

```
points = [(2, 2), (-10, 15), (10, 9), (2, -4)]  
points.sort() # сортируем список точек на месте  
print(points)
```

ВЫВОДИТ:

```
[(-10, 15), (2, -4), (2, 2), (10, 9)]
```


Встроенные функции, принимающие функции в качестве аргументов

Рассмотрим следующий код:

```
def compare_by_second(point) :  
    return point[1]  
  
def compare_by_sum(point) :  
    return point[0] + point[1]  
  
points = [(2, 3), (-10, 15), (10, 9), (2, -4)]  
print(sorted(points, key=compare_by_second))  
    # сортируем по второму значению кортежа  
print(sorted(points, key=compare_by_sum))  
    # сортируем по сумме кортежа
```

Результат:

[(2, -4), (2, 2), (10, 9), (-10, 15)]

[(2, -4), (2, 2), (-10, 15), (10, 9)]

Функции в качестве возвращаемых значений других функций

Объектная сущность функций позволяет и передавать их в качестве аргументов в другие функции, и возвращать одни функции из других.

То есть, функции могут быть результатом работы других функций, что позволяет писать генераторы функций, возвращающие функции в зависимости от передаваемых им аргументов.

Функции в качестве возвращаемых значений других функций

Пример 1: Рассмотрим код, где функция `generator()` возвращает функцию `hello()` в качестве результата своей работы.

```
def generator():  
    def hello():  
        print('Hello from function!')  
    return hello  
  
func = generator()  
func()
```



В Python можно определять функцию внутри функции, ведь функция это объект.



Что получим?

Результат:
Hello from function!

Функции в качестве возвращаемых значений других функций

Пример 2: Рассмотрим семейство функций — квадратных трехчленов.

Все эти функции имеют один и тот же вид

$$f(x) = ax^2 + bx + c.$$

Мы можем написать генератор функций, который по параметрам a , b , c вернет нам значение конкретного квадратного трехчлена

Функции в качестве возвращаемых значений других функций

Рассмотрим следующий код:

```
def generator_polynom(a, b, c):  
    def square_polynom(x):  
        return a*x**2 + b*x + c  
    return square_polynom  
  
f = generator_polynom(a=1, b=2, c=1)  
g = generator_polynom(a=2, b=0, c=-3)  
h = generator_polynom(a=-3, b=-10, c=50)  
  
print(f(1))  
print(g(2))  
print(h(-1))
```

Результат:

4
5
57

Функции в качестве возвращаемых значений других функций

Обратите внимание на то, что внутренняя функция `square_polynom()` использует параметры внешней функции `generator_square_polynom()`. Такую вложенную функцию называют замыканием.

Замыкания — вложенные функции, ссылающиеся на переменные, объявленные вне определения этой функции, и не являющиеся её параметрами.

Примечания

Примечание 1. Функция `sorted()` и списочный метод `sort()` помимо необязательного аргумента `key`, принимают еще аргумент `reverse`, который по умолчанию имеет значение `False`, что соответствует сортировке по возрастанию. Если значение `reverse` установить в `True`, произойдет сортировка по убыванию.

Примечание 2. Функции `max()` и `min()` возвращают первый максимальный или минимальный элемент, если таковых несколько.

Примечания

Примечание 3.



Что получим в каждом случае?

```
print(input())
```

Результат:
запросит у пользователя текст и тут же его напечатает, т.к. функция `input()` будет вызвана.

```
print(input)
```

Результат:
выдаст нам текстовое представление этой функции: строку `built-in function input`, которая поясняет, что `input` — встроенная функция языка.

Функции высшего порядка

Функции высшего порядка

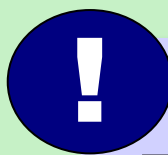
Функции, которые принимают или/и возвращают другие функции, называются **функциями высшего порядка**.

Например:

```
# функция высшего порядка, так как принимает функцию
def high_order_function(func):
    return func(3)

# обычные функции = функции первого порядка
def double(x):
    return 2*x

def add_one(x):
    return x + 1
```



Функция `high_order_function()` принимает другую функцию и возвращает результат её вызова с аргументом, равным 3.

Функции высшего порядка

функция высшего порядка, так как принимает функцию

```
def high_order_function(func):  
    return func(3)
```

обычные функции = функции первого порядка

```
def double(x):  
    return 2*x
```

```
def add_one(x):  
    return x + 1
```

```
print(high_order_function(double))
```

```
print(high_order_function(add_one))
```



Что получим?

Результат:

6
4

Функции высшего порядка для обработки набора данных

Часто функции высшего порядка используются для обработки наборов данных. Рассмотрим три важные функции высшего порядка:

- `map()`;
- `filter()`;
- `reduce()`.

В языке Python эти функции уже реализованы, однако для лучшего понимания их работы мы сначала напишем свои версии этих функций (`my_map`, `my_filter`, `my_reduce`) и уже после этого изучим встроенные реализации.

Функция `my_map()`

При работе со списками часто требуется применить одно и то же преобразование к каждому элементу. Можно написать цикл, содержащий нужное преобразование.

Например, для преобразования списка чисел в список их квадратов, код может выглядеть так:

```
def f(x):  
    return x**2  
  
old_list = [1, 2, 4, 9, 10, 25]  
new_list = []  
for i in old_list:  
    new_i = f(i)  
    new_list.append(new_i)  
print(old_list)  
print(new_list)
```



Что получим?

Результат:

[1, 2, 4, 9, 10, 25]

[1, 4, 16, 81, 100, 625]

Функция `my_map()`

Для других преобразований меняться будет только применяемая функция `f()`. Оптимизируем код, чтобы функция стала параметром.

```
def my_map(function, items):  
    result = []  
    for i in items:  
        new_i = function(i)  
        result.append(new_i)  
    return result
```

Функция `my_map()`

Теперь можно совершать преобразования, используя функцию высшего порядка `my_map()`.

```
def my_map(function, items):  
    ... # функция my_map() определена
```

```
def square(x):  
    return x**2
```

```
def cube(x):  
    return x**3
```

```
numbers = [1, 2, -3, 4, -5, 6, -9, 0]
```

```
strings = my_map(str, numbers)
```

```
abs_numbers = my_map(abs, numbers)
```

```
squares = my_map(square, numbers)
```

```
cubes = my_map(cube, numbers)
```

```
print(strings)
```

```
print(abs_numbers)
```

```
print(squares)
```

```
print(cubes)
```

Результат:

['1', '2', '-3', '4', '-5', '6', '-9', '0']

[1, 2, 3, 4, 5, 6, 9, 0]

[1, 4, 9, 16, 25, 36, 81, 0]

[1, 8, -27, 64, -125, 216, -729, 0]

Функция my_map(). Цепочки преобразований

Построим цепочку преобразований, несколько раз вызывая функцию my_map().

```
def my_map(function, items):  
    ... # функция my_map() определена как указано выше  
numbers = ['-1', '20', '3', '-94', '65', '6',  
           '-970', '8']  
new_numbers = my_map(abs, my_map(int, numbers))  
print(new_numbers)
```



Что получим?

Результат:
[1, 20, 3, 94, 65, 6, 970, 8]

Функция `my_filter()`

Другая популярная задача при работе со списками: отобрать часть элементов списка по определенному критерию.

Функция - критерий, которая возвращает значение `True` или `False` , называется **предикатом**.

Реализация такой функции может выглядеть так:

```
def my_filter(function, items):  
    result = []  
    for i in items:  
        if function(i):  
            # если function вернула True  
            result.append(i)  
    return result
```

Функция my_filter()

Пример 1:

Из исходного списка чисел получить список с элементами, большими 10:

```
def my_filter(function, items):  
    ... # функция my_filter()  
        #определена как указано выше  
def is_greater10(num):  
    return num > 10  
numbers = [12, 2, -30, 48, 51, -60, 19, 10, 13]  
large_numbers = my_filter(is_greater10, numbers)  
print(large_numbers)
```



Что получим?

Результат:
[12, 48, 51, 19, 13]

Функция my_filter()

Пример 2:

```
def my_filter(function, items):
```

```
    ... # функция my_filter()
```

```
        #определена как указано выше
```

```
def is_odd(num):
```

```
    return num % 2
```

```
def is_word_long(word):
```

```
    return len(word) > 6
```

```
numbers = list(range(15))
```

```
words = ['В', 'новом', 'списке', 'останутся',  
         'только', 'длинные', 'слова']
```

```
odd_numbers = my_filter(is_odd, numbers)
```

```
large_words = my_filter(is_word_long, words)
```

```
print(odd_numbers)
```

```
print(large_words)
```



Что получим?

Результат:

[1, 3, 5, 7, 9, 11, 13]

['останутся', 'длинные']

Функция `my_reduce()`

Реализованные функции `my_map()` и `my_filter()` работали с отдельными элементами списка независимо. Но встречаются циклы с агрегацией результата — формированием одного результирующего значения при комбинации элементов с использованием аргумента-аккумулятора.

Типичные примеры агрегации — сумма всех элементов списка или их произведение.

Функция `my_reduce()`

Например:

```
numbers = [1, 2, 3, 4, 5]
total = 0
product = 1
for num in numbers:
    total += num
    product *= num
print(total)
print(product)
```

?

Что вычисляет код?

?

Что получим?

Результат:

15
120

Функция `my_reduce()`

С точки зрения математики сумма $1+2+3+4+5$ может быть выражена как:

$$((((0+1)+2)+3)+4)+5).$$

Ноль здесь тот самый аккумулятор, точнее его начальное значение. Он не добавляет к сумме ничего, поэтому может служить отправной точкой. А еще будет результатом, если входной список пуст.

Этот цикл будет выглядеть одинаково практически во всех случаях. Меняться будет только начальное значение аккумулятора (0 для суммы, 1 для произведения и т.д.) и операция, которая комбинирует элемент и аккумулятор. Обобщим данный код

Функция my_reduce()

```
def my_reduce(operation, items, initial_value):  
    acc = initial_value  
    for i in items:  
        acc = operation(acc, i)  
    return acc  
  
def add(x, y):  
    return x+y  
  
def mult(x, y):  
    return x*y  
  
numbers = [1, 2, 3, 4, 5]  
total = my_reduce(add, numbers, 0)  
product = my_reduce(mult, numbers, 1)  
print(total)  
print(product)
```



Что получим?

Результат:
15
120

Примечание

Мы с вами реализовали три функции:

- `my_map()`: преобразование элементов списка;
- `my_filter()`: фильтрация элементов списка;
- `my_reduce()`: агрегация элементов списка.

Каждая функция имеет меньшую мощность, чем цикл `for`. Цикл `for` позволяет гибко управлять процессом итерации, мы можем использовать даже команды `break` и `continue`. Возникает резонный вопрос: зачем нужны отдельные функции, когда можно обойтись циклом?

Во-первых, такие функции — **часть функционального подхода**.

Во-вторых, каждая такая функция делает единственную работу, что значительно упрощает рассуждение о коде, его чтение и написание.

Встроенные функции высшего порядка

Встроенная функция `map()`

Встроенная функция `map()` имеет сигнатуру:

`map(func, *iterables)`

В отличие от самописной `my_map()`, встроенная функция `map()` может принимать сразу несколько последовательностей, переменное количество аргументов.

В качестве параметра `func` указывается функция, которой будет передаваться текущий элемент последовательности. Внутри функции `func` необходимо вернуть новое значение.

Встроенная функция map()

Например, прибавим к каждому элементу списка число 7:

```
def plus_7(num):  
    return num + 7  
  
numbers = [1, 2, 3, 4, 5, 6]  
  
# используем встроенную функцию map()  
new_numbers = map(plus_7, numbers)  
print(new_numbers)
```

Результат:
<map object at 0x0000017520777190>

! Выведен не список, а специальный объект!

Такой объект в Python называют итератор. Он похож на список тем, что его можно перебирать циклом for, то есть итерировать.

Встроенная функция map()

Выполним итерацию циклом for:

```
def plus_7(num):  
    return num + 7  
  
numbers = [1, 2, 3, 4, 5, 6]  
  
# используем встроенную функцию map()  
new_numbers = map(plus_7, numbers)  
for num in new_numbers:  
    print(num)
```



Что получим?



Как получить
из итератора
список?

Результат:

8
9
10
11
12
13

Встроенная функция map()

Чтобы получить из итератора список, нужно воспользоваться функцией `list()`:

```
def plus_7(num):  
    return num + 7  
numbers = [1, 2, 3, 4, 5, 6]  
# используем встроенную функцию map()  
new_numbers = list(map(plus_7, numbers))  
print(new_numbers)
```

Результат:
[8, 9, 10, 11, 12, 13]

Встроенная функция `map()`

Функции `map()` можно передать несколько последовательностей.

В этом случае в функцию обратного вызова `func` будут передаваться сразу несколько элементов, **расположенных в последовательностях на одинаковых позициях.**

Встроенная функция map()

Пример 1. Выполним суммирование элементов трех списков:

```
def func(elem1, elem2, elem3):  
    return elem1 + elem2 + elem3  
  
numbers1 = [1, 2, 3, 4, 5]  
numbers2 = [10, 20, 30, 40, 50]  
numbers3 = [100, 200, 300, 400, 500]  
  
new_numbers = list(map(func, numbers1,  
                        numbers2, numbers3))  
  
print(new_numbers)
```



А если в списках
разное число
элементов?

Результат:
[111, 222, 333, 444, 555]

Встроенная функция map()

Если в последовательностях разное количество элементов, то последовательность с минимальным количеством элементов становится ограничителем.

Например:

```
def func(elem1, elem2, elem3):  
    return elem1 + elem2 + elem3  
numbers1 = [1, 2, 3, 4]  
numbers2 = [10, 20]  
numbers3 = [100, 200, 300, 400, 500]  
new_numbers = list(map(func, numbers1,  
                        numbers2, numbers3))  
print(new_numbers)
```

Результат:
[111, 222]

Встроенная функция map()

Пример 2. Использование встроенной функции map(), которой передано две последовательности:

```
circle_areas = [3.56773, 5.57668, 4.31914]
# округлим числа до 1 знака после запятой
result1 = list(map(round, circle_areas, [1]*3))
# округлим числа до 1,2,3 знаков после запятой
result2 = list(map(round, circle_areas,
                    range(1, 4)))

print(result1)
print(result2)
```

Результат:

[3.6, 5.6, 4.3]

[3.6, 5.58, 4.319]

Встроенная функция `map()`

Встроенная функция `map()` реализована очень гибко. В качестве последовательностей мы можем использовать: списки, строки, кортежи, множества, словари.

Встроенная функция `filter()`

Встроенная функция `filter()` имеет сигнатуру
`filter(func, iterable)`

В отличие от самописной функции `my_filter`, она может принимать любой итерируемый объект (список, строку, кортеж, и т.д.).

В качестве параметра `func` указывается ссылка на функцию, которой будет передаваться текущий элемент последовательности. Внутри функции `func` необходимо вернуть значение `True` или `False`.

Встроенная функция filter()

Например, удалим все отрицательные значения из списка:

```
def func(elem):  
    return elem >= 0  
numbers = [-1, 2, -3, 4, 0, -20, 10]  
# преобразуем итератор в список  
positive_numbers = list(filter(func, numbers))  
print(positive_numbers)
```

Результат:

[2, 4, 0, 10]



Функция filter() как и функция map() возвращает не список, а специальный объект, который называется **итератором**.

Встроенная функция filter()

Встроенной функции `filter()` можно в качестве первого параметра `func` передать значение `None`. В таком случае каждый элемент последовательности будет проверен на соответствие значению `True`. Если элемент в логическом контексте возвращает значение `False`, то он не будет добавлен в возвращаемый результат.

```
true_values = filter(None, [1, 0, 10, '', None,
                             [], [1, 2, 3], ()])
for value in true_values:
    print(value)
```

Результат:

1
10
[1, 2, 3]

! Значения списка: 0, "", None, [], ()
позиционируются как False, а
значения 1, 10, [1, 2, 3] как True

Функция `reduce()`

Для использования функции `reduce()` необходимо подключить специальный модуль `functools`. Функция `reduce()` имеет сигнатуру

`reduce(func, iterable, initializer=None)`

Если начальное значение не установлено, то в его качестве используется первое значение из последовательности **`iterable`**.



Функция `reduce()` может принимать любой итерируемый объект

Функция reduce()

Например:

```
from functools import reduce
def func(a, b):
    return a + b
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
# в качестве начального значения 0
total = reduce(func, numbers, 0)
print(total)
```

Результат:
55

Можно вызвать функцию reduce() и так:

```
total = reduce(func, numbers)
# в качестве начального значения первый
# элемент списка numbers
```

Модуль operator

Чтобы не писать каждый раз функции, определяющие такие стандартные математические операции как сумма или произведение:

```
def sum_func(a, b):  
    return a + b  
def mult_func(a, b):  
    return a * b
```

Можно использовать уже реализованные функции из модуля operator.

Модуль operator

Список функций из модуля operator:

Операция	Синтаксис	Функция
Сложение	$a + b$	<code>add(a, b)</code>
Деление	a / b	<code>truediv(a, b)</code>
Целочисленное деление	$a // b$	<code>floordiv(a, b)</code>
Возведение в степень	$a ** b$	<code>pow(a, b)</code>
Остаток от деления	$a \% b$	<code>mod(a, b)</code>
Произведение	$a * b$	<code>mul(a, b)</code>

Модуль operator

Список функций из модуля operator:

Операция	Синтаксис	Функция
Смена знака	$-a$	<code>neg(a)</code>
Вычитание	$a - b$	<code>sub(a, b)</code>
Проверка на неравенство $<$	$a < b$	<code>lt(a, b)</code>
Проверка на неравенство $\leq$	$a \leq b$	<code>le(a, b)</code>
Проверка на равенство	$a == b$	<code>eq(a, b)</code>
Проверка на неравенство	$a != b$	<code>ne(a, b)</code>
Проверка на неравенство $\geq$	$a \geq b$	<code>ge(a, b)</code>
Проверка на неравенство $>$	$a > b$	<code>gt(a, b)</code>

Модуль operator

Пример 1:

```
from operator import *  
print(add(10, 20))  
print(floordiv(20, 3))  
print(neg(9))  
print(lt(2, 3))  
print(lt(10, 8))  
print(eq(5, 5))  
print(eq(5, 9))
```

Результат:

30
6
-9
True
False
True
False

Примечания

Примечание 1. Итераторы – важная концепция языка Python.

Нужно помнить:

- ❑ итераторы можно обойти циклом `for`;
- ❑ итератор можно преобразовать в список или кортеж, с помощью функций `list()` и `tuple()`;
- ❑ итератор можно распаковать с помощью `*`

Например:

```
numbers = [1, 10, -9, 8, 9, 345, -32, -89, 2]
map_obj = map(abs, numbers)
print(*map_obj)
```

Результат:
1 10 9 8 9 345 32 89 2

Примечания

Примечание 2.

Если нам нужны строковые методы в виде функций, мы можем получить их через название типа str.

Например:

```
pets = ['alfred', 'tabitha', 'william']  
chars = ['x', 'y', '2', '3', 'a']  
uppered_pets = list(map(str.upper, pets))  
capitalized_pets = list(map(str.capitalize, pets))  
only_letters = list(filter(str.isalpha, chars))  
print(uppered_pets)  
print(capitalized_pets)  
print(only_letters)
```

Результат:

['ALFRED', 'TABITHA', 'WILLIAM']

['Alfred', 'Tabitha', 'William']

['x', 'y', 'a']

Примечания

Примечание 3.

Итератор можно перебрать только один раз за время выполнения программы. Он "исчерпывает" себя.

Например, если из итератора создать список, то дальше в программе сам итератор уже не получится использовать в том же цикле, т.к. он будет возвращать пустые значения.

Или наоборот, если прошлись по итератору циклом, то из него нельзя будет построить список.

Примечания

Например:

```
chars = ['1', '2', '3', '4']
itr = map(int, chars)

a = list(itr)    # получит список [1, 2, 3, 4]
b = list(itr)    # получит пустой список
c = list(itr)    # так же получит пустой список
print(a, b, c)
for item in itr:
    # данный цикл не запустится,
    # т.к. итератор уже пустой
    print(item)
```

Результат:

[1, 2, 3, 4] [] []

Примечания

Примечание 4. Итератор сам не содержит элементы, а обращается с элементами другого объекта

Например:

```
seq = ['1', '2', '3', '4', '5']  
str_to_int = map(int, seq)  
del seq[2]  
print(list(str_to_int))
```



Что получим?

Результат:
[1, 2, 4, 5]

Анонимные функции

Анонимные функции

Помимо стандартного определения функции, состоящего из ее заголовка с ключевым словом `def` и блока кода – тела функции, в Python можно создавать короткие однострочные функции с использованием оператора **`lambda`**. Это анонимные функции или лямбда-функции.



Анонимные функции (лямбда-функции) – функции с телом, но без имени.

Общий формат определения анонимной функции:

```
lambda список_параметров: выражение
```

Анонимные функции

Следующие два определения функций эквивалентны:

```
# стандартное объявление функции
```

```
def standard_function(x):  
    return x*2
```

```
# объявление анонимной функции
```

```
lambda_function = lambda x: x*2
```



Параметры анонимных функций, в отличие от обычных, не нужно заключать в скобки.

Анонимные функции

Например:

?

Что получим?

```
# функция без параметров
f1 = lambda: 10 + 20

# функция с двумя параметрами
f2 = lambda x, y: x + y

# функция с тремя параметрами
f3 = lambda x, y, z: x + y + z

print(f1())
print(f2(5, 10))
print(f3(5, 10, 30))
```

Результат:

30

15

45

Анонимные функции

Когда применение анонимных функций оправдано:

- ☐ однократное использование функции;
- ☐ передача функций в качестве аргументов другим функциям;
- ☐ возвращение функции в качестве результата другой функции.

Однократное использование функции

Для сортировки, отличной от стандартной приходится создавать отдельные функции-компараторы для сравнения элементов:

```
def compare_by_second(point):  
    return point[1]
```

```
def compare_by_sum(point):  
    return point[0] + point[1]
```

```
points = [(2, 3), (-10, 15), (10, 9), (2, -4)]
```

```
print(sorted(points, key=compare_by_second))
```

```
# сортируем по второму значению кортежа
```

```
print(sorted(points, key=compare_by_sum))
```

```
# сортируем по сумме
```

[(2, -4), (2, 2), (10, 9), (-10, 15)]

[(2, -4), (2, 2), (-10, 15), (10, 9)]

Однократное использование функции

Очевидно, что такие функции как `compare_by_second()` и `compare_by_sum()` не особо нужны вне контекста сортировки, поэтому логично их заменить на анонимные функции:

```
points = [(2, 3), (-10, 15), (10, 9), (2, -4)]
print(sorted(points,
              key=lambda point: point[1]))
# сортируем по второму значению кортежа
print(sorted(points,
              key=lambda point: point[0] + point[1]))
# сортируем по сумме элементов кортежа
```

Передача анонимных функций в качестве аргументов другим функциям

Рассмотрим использование анонимных функций в качестве аргументов функций высшего порядка `map()`, `filter()` и `reduce()`.

Пример1:

?

Что получим?

```
numbers = [1, 2, 3]
new_numbers1 = list(map(lambda x: x+1, numbers))
new_numbers2 = list(map(lambda x: x*2, numbers))
new_numbers3 = list(map(lambda x: x**2,
numbers))
print(new_numbers1)
print(new_numbers2)
print(new_numbers3)
```

Результат:

[2, 3, 4]

[2, 4, 6]

[1, 4, 9]

Передача анонимных функций в качестве аргументов другим функциям

Пример 2:



Что получим?

```
strings = ['a', 'b', 'c']  
numbers = [3, 2, 1]  
  
new_strings = list(map(lambda x, y: x*y, strings,  
                        numbers))  
  
print(new_strings)
```

Результат:
['aaa', 'bb', 'c']

Передача анонимных функций в качестве аргументов другим функциям

Пример 3:

?

Что получим?

```
numbers = [-1, 2, -3, 14, 0, -20]

positive_numbers = list(filter(lambda x: x > 0,
                                numbers))

large_numbers = list(filter(lambda x: x > 5,
                             numbers))

even_numbers = list(filter(lambda x: x % 2 == 0,
                            numbers))

print(positive_numbers)
print(large_numbers)
print(even_numbers)
```

Результат:

[2, 14]

[14]

[2, 14, 0, -20]

Передача анонимных функций в качестве аргументов другим функциям

Пример 4:

?

Что получим?

```
words = ['march' , 'october' , 'november']

new_words1 = list(filter(lambda w: len(w) > 6,
                          words))

new_words2 = list(filter(lambda w: 'a' in w,
                          words))

print(new_words1)
print(new_words2)
```

Результат:

['october', 'november']
['march']

Передача анонимных функций в качестве аргументов другим функциям

Пример 5:

?

Что получим?

```
from functools import reduce
words = ['march' , 'october' , 'november']
numbers = [1, 3, 5]
summa = reduce(lambda x, y: x + y, numbers, 0)
product = reduce(lambda x, y: x * y, numbers, 1)
sentence = reduce(lambda x, y: x + ', ' + y,
                  words, 'January')

print(summa)
print(product)
print(sentence)
```

Результат:

9

15

January, march, october, november

Возвращение функции в качестве результата другой функции

Анонимные функции могут быть результатом работы других функций.

```
def generator_square_polynom(a, b, c):  
    def square_polynom(x):  
        return a*x**2 + b*x + c  
    return square_polynom
```

Такой код можно переписать так:

```
def generator_square_polynom(a, b, c):  
    return lambda x: a*x**2 + b*x + c
```



Анонимные функции являются замыканиями: возвращаемая функция запоминает значения переменных a , b , c из внешнего окружения.

Условный оператор в теле анонимной функции

В теле анонимной функции не получится выполнить несколько действий и не получится использовать многострочные конструкции вроде циклов `for` и `while`.

Но можно использовать тернарный условный оператор.

Общий вид тернарного условного оператора в теле анонимной функции выглядит так:

```
значение1 if условие else значение2
```

Если условие истинно, возвращается значение1 ,
если нет – значение2.

Условный оператор в теле анонимной функции

Например:

?

Что получим?

```
numbers = [-2, 0, 1, 2]
```

```
result = list(map(lambda x: 'even' if x % 2 == 0  
                    else 'odd', numbers))
```

```
print(result)
```

Результат:

['even', 'even', 'odd', 'even']

Передача аргументов в анонимную функцию

Как и обычные функции, определенные с помощью ключевого слова `def`, анонимные функции поддерживают все способы передачи аргументов:

- ☐ позиционные аргументы;
- ☐ именованные аргументы;
- ☐ переменный список позиционных аргументов (`*args`);
- ☐ переменный список именованных аргументов (`**kwargs`);
- ☐ обязательные аргументы (`*`).

Передача аргументов в анонимную функцию

Например:

?

Что получим?

```
f1 = lambda x, y, z: x + y + z
f2 = lambda x, y, z=3: x + y + z
f3 = lambda *args: sum(args)
f4 = lambda **kwargs: sum(kwargs.values())
f5 = lambda x, *, y=0, z=0: x + y + z
```

```
print(f1(1, 2, 3))
print(f2(1, 2))
print(f2(1, y=2))
print(f3(1, 2, 3, 4, 5))
print(f4(one=1, two=2, three=3))
print(f5(1))
print(f5(1, y=2, z=3))
```

Результат:

6

6

6

15

6

1

6

Ограничения и особенности анонимных функций

- ❑ Анонимная функция может содержать только выражение, и не может включать в свое тело операторы (кроме условного оператора);
- ❑ В теле анонимной функции такие операторы, как `return` или `pass` вызовут исключение `SyntaxError`;
- ❑ Анонимная функция пишется как одна строка исполнения;
- ❑ В Python анонимные функции — лишь сокращенная запись функции, поэтому:

```
f = lambda x: x + 1  
print(type(f)) #выводит <class 'function'>
```



Анонимные функции имеют такой же тип, как и обычные функции.

**Встроенные функции any(),
all(), zip(), enumerate()**

Функция `all()`

Встроенная функция `all()` возвращает значение `True`, если все элементы переданной ей последовательности (итерируемого объекта) истинны (приводятся к значению `True`), и `False` в противном случае.

Сигнатура функции следующая:

`all(iterable)`

В качестве `iterable` может выступать любой итерируемый объект:

- ☐ список;
- ☐ кортеж;
- ☐ строка;
- ☐ множество;
- ☐ словарь.

Функция all()

Приведенный ниже код:

```
print(all([True, True, True]))  
#возвращает True, так как все значения  
#списка равны True  
print(all([True, True, False]))  
#возвращает False, так как не все значения  
#списка равны True
```

Выводит:
True
False

В Python все следующие значения приводят к значению **False**:

- ☐ константы None и False;
- ☐ нули всех числовых типов данных: 0, 0.0, 0j, Decimal(0), Fraction(0, 1);
- ☐ пустые коллекции: "", (), [], {}, set(), range(0).

Функция all()

Например:

?

Что получим?

```
print(all([1, 2, 3]))  
print(all([1, 2, 3, 0, 5]))  
print(all([True, 0, 1]))  
print(all(['', 'red', 'green']))  
print(all({0j, 3+4j}))
```

Результат:

True

False

False

False

False

Функция all()

При работе со словарями функция all() проверяет на соответствие параметрам True ключи словаря, а не их значения.

```
dict1 = {0: 'Zero', 1: 'One', 2: 'Two'}  
dict2 = {'Zero': 0, 'One': 1, 'Two': 2}  
  
print(all(dict1))  
print(all(dict2))
```

?

Что получим?

Результат:
False
True

Функция all()

Если переданный итерируемый объект пустой, то функция all() возвращает значение True

```
print(all([]))  
print(all(()))  
print(all(''))  
print(all([], []))
```

?

Что получим?

Результат:

True

True

True

False

Функция `any()`

Встроенная функция `any()` возвращает значение `True`, если хотя бы один элемент переданной ей последовательности (итерируемого объекта) является истинным (приводится к значению `True`), и `False` в противном случае.

Сигнатура функции следующая:

`any(iterable)`

В качестве `iterable` может выступать любой итерируемый объект

Функция any()

Например:

```
print(any([False, True, False]))  
# возвращает True, так как есть хотя бы один  
# элемент, равный True  
print(any([False, False, False]))  
# возвращает False, так как нет элементов,  
# равных True
```

Результат:
True
False

Функция any()

Например:

```
print(any([0, 0, 0]))  
print(any([0, 1, 0]))  
print(any([False, 0, 1]))  
print(any(['', [], 'green']))  
print(any({0j, 3+4j, 0.0}))
```

?

Что получим?

Результат:

False

True

True

True

True

Функция any()

При работе со словарями функция any() проверяет на соответствие True ключи словаря, а не их значения.

```
dict1 = {0: 'Zero'}  
dict2 = {'Zero': 0, 'One': 1}  
  
print(any(dict1))  
print(any(dict2))
```

?

Что получим?

Результат:
False
True

Функция any()

Если переданный объект пуст, то функция any() возвращает значение False

```
print(any([]))  
print(any(()))  
print(any(''))  
print(any([], []))
```



Что получим?

Результат:

False

False

False

False

Использование функций `all()` и `any()` вместе с функцией `map()`



Что получим?

Пример 1:

Все ли элементы списка `numbers` больше 10?

```
numbers = [17, 56, 231, 45, 5, 89, 91, 11, 19]
result = all(map(lambda x: True if x > 10
                  else False, numbers))

if result:
    print('Все числа больше 10')
else:
    print('Хотя бы одно число меньше или равно 10')
```

Результат:

Хотя бы одно число меньше или равно 10

Использование функций `all()` и `any()` вместе с функцией `map()`



Что получим?

Пример 2:

Список содержит хотя бы одно четное число?

```
numbers = [17, 91, 78, 55, 89, 99, 11, 19]
result = any(map(lambda x: x % 2 == 0, numbers))
if result:
    print('Хотя бы одно число четное')
else:
    print('Все числа нечетные')
```

Результат:

Хотя бы одно число четное

Функция enumerate()

Встроенная функция `enumerate()` итератор, элементами которого являются кортежи из индекса элемента и самого элемента переданной ей последовательности (итерируемого объекта).

Сигнатура функции следующая:

`enumerate(iterable, start)`

В качестве `iterable` может выступать любой итерируемый объект

С помощью необязательного параметра `start` можно задать начальное значение индекса. По умолчанию значение параметра `start = 0`, то есть счет начинается с нуля.

Функция enumerate()

Например:

```
colors = ['red', 'green', 'blue']  
pairs = enumerate(colors)  
for i in pairs:  
    print(i)
```

Результат:

(0, 'red')

(1, 'green')

(2, 'blue')

- ❗ Функция enumerate() возвращает итератор!
- ❑ итераторы можно обойти циклом for;
- ❑ итератор можно преобразовать в список или кортеж, с помощью функций list() и tuple();
- ❑ итератор можно распаковать

Функция enumerate()

Варианты работы с итератором:

Преобразуем в список с помощью функции list().

```
print(list(pairs))
```

Результат:

`[(0, 'red'), (1, 'green'), (2, 'blue')]`

Используем распаковку кортежей при итерировании с помощью цикла for:

```
for index, item in pairs:  
    print(index, item)
```

Результат:

0 red

1 green

2 blue

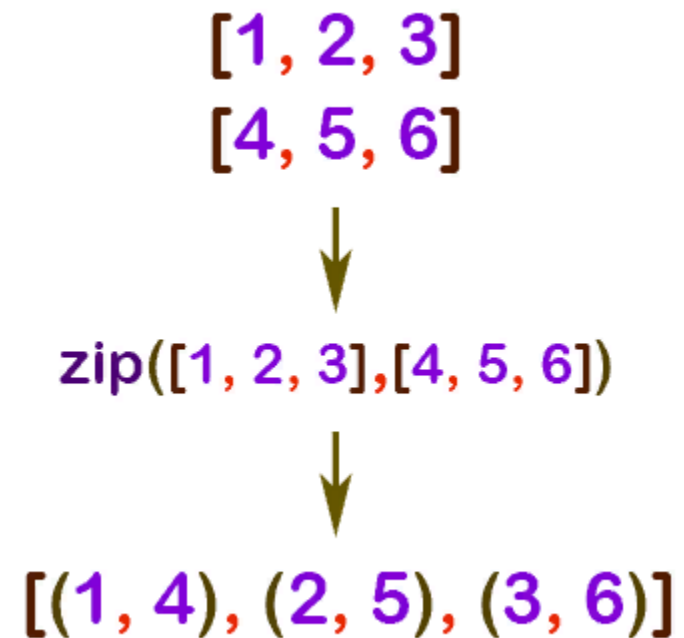
Функция zip()

Встроенная функция zip() объединяет отдельные элементы из каждой переданной ей последовательности (итерируемого объекта) в кортежи.

Сигнатура функции следующая:

zip(*iterables)

В качестве iterable может выступать любой итерируемый объект



Функция zip()

Например:

```
numbers = [1, 2, 3]
words = ['one', 'two', 'three']
for pair in zip(numbers, words):
    print(pair)
```

Результат:
(1, 'one')
(2, 'two')
(3, 'three')

! Функция zip() возвращает итератор!

Функция zip()

Можно передавать функции `zip()` сколько угодно итерируемых объектов.

Например:

```
numbers = [1, 2, 3]
words = ['one', 'two', 'three']
romans = ['I', 'II', 'III']

result = zip(numbers, words, romans)
print(list(result))
```

Результат:

```
[(1, 'one', 'I'), (2, 'two', 'II'), (3, 'three', 'III')]
```

Функция zip()

Если функции zip() передать итерируемые объекты, имеющие разную длину, то объект с наименьшим количеством элементов определяет итоговую длину:

```
numbers = [1, 2, 3, 4]
words = ['one', 'two']
romans = ['I', 'II', 'III']

result = zip(numbers, words, romans)
print(list(result))
```

Результат:
[(1, 'one', 'I'), (2, 'two', 'II')]

Частые сценарии использования функции `zip()`

Сценарий 1. Функция `zip()` удобна для создания словарей, когда ключи и значения находятся в разных списках.

```
keys = ['name', 'age', 'gender']  
values = ['Timur', 28, 'male']  
  
info = dict(zip(keys, values))  
print(info)
```

Результат:

```
{'name': 'Timur', 'age': 28, 'gender': 'male'}
```

Частые сценарии использования функции `zip()`

Сценарий 2. Функция `zip()` удобна для одновременного (параллельного) итерирования сразу по нескольким коллекциям.

```
name = ['Timur', 'Ruslan', 'Rustam']  
age = [28, 21, 19]  
  
for x, y in zip(name, age):  
    print(x, y)
```

Результат:
Timur 28
Ruslan 21
Rustam 19

Частые сценарии использования функции `zip()`

Сценарий 3 . Одновременное использование функции `zip()` и `enumerate()`

```
list1 = ['a1', 'a2', 'a3']  
list2 = ['b1', 'b2', 'b3']  
for index, (item1, item2) in enumerate(zip(list1,  
                                           list2)):  
    print(index, item1, item2)
```

Результат:

0 a1 b1

1 a2 b2

2 a3 b3

Итераторы. Встроенные функции `iter()` и `next()`.

Итерируемые объекты

В языке Python под итерируемым объектом подразумевают объект, который можно итерировать, то есть проходиться по нему, перебирая каждый элемент раз за разом.

К примеру, списки (тип `list`), строки (тип `str`), кортежи (тип `tuple`), множества (тип `set`), словари (тип `dict`) являются итерируемыми, поскольку мы можем перебирать каждый элемент этих объектов.

В Python, существует два типа итерируемых объектов:

- ☐ итераторы
- ☐ коллекции и последовательности

Итераторы. Функция `next()`

Итератор — специальный объект, который выдает свои элементы по одному за раз.

Если итератор передать во встроенную функцию `next()`, то эта функция вернет его следующий элемент. При этом сам итератор также сдвинется на следующий элемент. При следующем вызове функция `next()` вернет следующий элемент и т.д.

Если же в итераторе элементов больше не осталось, то вызов функции `next()` приведет к возникновению исключения **`StopIteration`**.

Коллекции и последовательности

Коллекция — объект, хранящий набор значений одного или различных типов, позволяющий обращаться к этим значениям, а также применять специальные функции и методы, зависящие от типа коллекции.

Среди коллекций можно выделить те, элементы которых пронумерованы индексами и расположены в строгом порядке. Такие коллекции называются **последовательностями**.

Например, списки, строки и кортежи являются последовательностями, а множества и словари нет.

Функция `iter()`

Коллекции не являются итераторами, но позволяют создать итератор на своей основе.

Получить итератор можно из любого итерируемого объекта, для этого нужно передать итерируемый объект во встроенную функцию **`iter()`**.



Встроенные функции `iter()` и `next()`

Например:

```
numbers = [1, 2, 3]
# создаем итератор на основании списка
iterator = iter(numbers)
# запрашиваем и печатаем первый элемент итератора
print(next(iterator))
# запрашиваем и печатаем второй элемент итератора
print(next(iterator))
# запрашиваем и печатаем третий элемент итератора
print(next(iterator))
```

Результат:

1
2
3

Отличие последовательностей от итераторов

Основная разница между последовательностями и итераторами, заключается в том, что в последовательностях элементы пронумерованы индексами, начиная от нуля. Мы можем обратиться к конкретному элементу таких объектов по индексу.

В итераторах мы можем лишь последовательно запрашивать следующий элемент.

ОТЛИЧИЕ ПОСЛЕДОВАТЕЛЬНОСТЕЙ ОТ ИТЕРАТОРОВ

Например:

```
numbers = [1, 2, 3, 4]
letters = ('a', 'b', 'c')
language = 'python'
iterator = iter(letters)
print(numbers[3])
print(letters[1])
print(language[4])
print(iterator[1])
```

Результат:

4

b

o

TypeError: 'tuple_iterator' object is not subscriptable

Преимущества итераторов

Основными преимуществами использования итераторов являются:

- ❑ однотипность работы с объектами разных типов
- ❑ экономия потребляемой памяти
- ❑ комбинация множества итераторов для создания понятной и читабельной программы

Однотипность работы с объектами разных типов

Итераторы позволяют разным объектам «притворяться» одинаковыми. Списки, кортежи, строки, множества, словари, объекты типа `range` имеют разные типы, но мы можем использовать любой из этих объектов:

- ☐ в цикле `for`
- ☐ в функциях высшего порядка `map()`, `filter()`, `reduce()`, `reversed()` и т.д.
- ☐ для проверки наличия некоторого значения с помощью оператора принадлежности `in`
- ☐ для распаковки элементов с помощью `*`

Однотипность работы с объектами разных типов

Цикл `for` в Python работает по следующему принципу:

- ❑ создает итератор на основе итерируемого объекта
- ❑ запрашивает очередной элемент из итератора с помощью функции `next()` и передает его в выполняемый блок кода (тело цикла)
- ❑ останавливается при получении исключения `StopIteration`

Благодаря этому, в цикл `for` можно передать и список, и кортеж, и строку, и объект типа `range`, и многие другие объекты, которые имеют свои итераторы.

Экономия потребляемой памяти

Объекты типа `range` являются итерируемыми объектами.

Цикл `for` создает на основе объекта `range` итератор, у которого запрашивает элементы по одному, пока не будет достигнут конец последовательности чисел.

Объект типа `range` не хранит весь набор чисел. Он создает новое число только тогда, когда оно потребуется, при этом старые значения не хранятся.

Размер объектов `range` не зависит от количества чисел, которые предполагается перебрать, ведь нужно помнить только начальное и конечное значения последовательности, шаг и текущее значение.

Экономия потребляемой памяти



Функция `getsizeof` модуля `sys` возвращает размер переданного объекта в байтах

Например:

```
from sys import getsizeof
numbers1 = range(5)
numbers2 = range(100000)
numbers3 = range(1000000000000000)

print(getsizeof(numbers1))
print(getsizeof(numbers2))
print(getsizeof(numbers3))
```

Результат:

48

48

48



Все объекты `range` имеют один и тот же размер в памяти — 48 байт. Такой подход позволяет создавать "большие" итераторы (даже бесконечные), не занимая много памяти.

Экономия потребляемой памяти

Можно преобразовать любой итерируемый объект в список. Однако при таком преобразовании все элементы итерируемого объекта будут записаны в память.

```
from sys import getsizeof
numbers1 = list(range(5))
numbers2 = list(range(100000))
print(getsizeof(numbers1))
print(getsizeof(numbers2))
```

Результат:

96

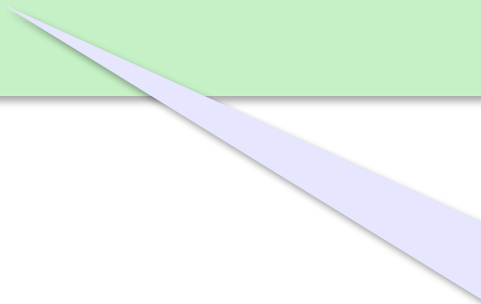
800056

Экономия потребляемой памяти

Преобразование итерируемого объекта в список не всегда заканчивается удачно.

```
from sys import getsizeof
numbers3 = list(range(1000000000000000))

print(getsizeof(numbers3))
```



Результат:
MemoryError

Примечания:

Примечание 1.

Встроенной функции `next()` можно передать второй аргумент, который будет возвращен вместо возбуждения исключения `StopIteration`, если в итераторе больше не осталось элементов.

```
nums = iter([1, 2, 3])  
print(next(nums))  
print(next(nums))  
print(next(nums))  
print(next(nums, -1))
```

Результат:

1
2
3
-1

Примечания:

Примечание 2.

Функция `len()` не применима к итераторам

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
evens = filter(lambda num: num % 2 == 0, numbers)
print(len(evens))
```

Результат:

`TypeError: object of type 'filter' has no len()`